Seminar in Software Engineering

HW1 bonus Workshop – Interval Analysis

READ THE ENTIRE ASSIGNMENT BEFORE STARTING.

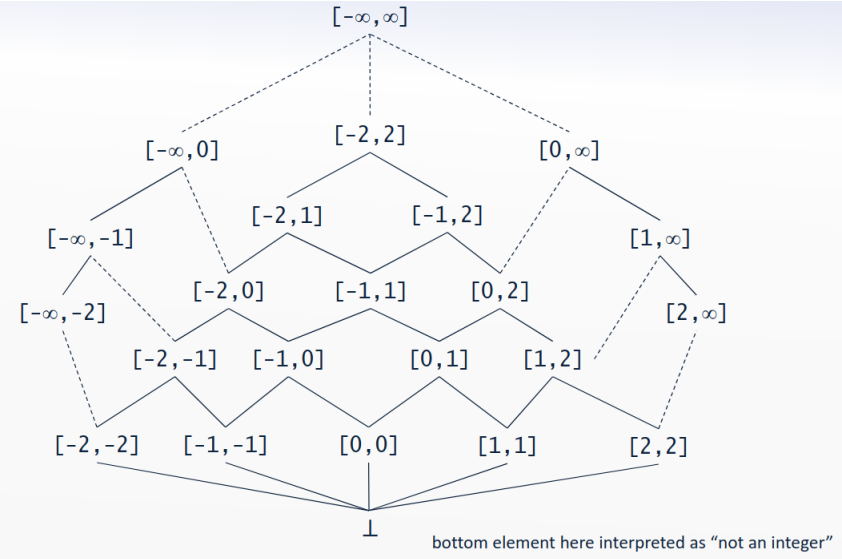
Deadline: November 30, 2026 **(15:59) (no extensions)**

Introduction to Interval Analysis

Interval Analysis is a kind of static analysis that abstracts the values of variables as numeric intervals. Instead of tracking exact values or properties such as "zero" or "non-zero", this analysis keeps information about the possible ranges of values that integer variables may take during program execution.

The Interval Lattice

The abstract domain of interval analysis consists of intervals of the form $[a, b]$ where $a, b \in \mathbb{Z} \cup \{-\infty, +\infty\}$ and $a \leq b$. The element $\bot$ (bottom) represents the empty set (an uninitialized variable or unreachable code).



In this lattice:

- $\bot$ is the smallest element, representing "no possible value"
- $[-\infty, +\infty]$ is the largest element, representing "any integer value"
- More specific intervals are at lower levels of the lattice
- The order relation $\sqsubseteq$ is given by set inclusion: $[a_1, b_1] \sqsubseteq [a_2, b_2]$ if $a_2 \leq a_1$ and $b_1 \leq b_2$

Operations on Intervals

The arithmetic operations are defined so that they preserve the soundness of the analysis:

- **Addition:** $[a_1, b_1] + [a_2, b_2] = [a_1 + a_2, b_1 + b_2]$
- **Subtraction:** $[a_1, b_1] - [a_2, b_2] = [a_1 - b_2, b_1 - a_2]$
- **Multiplication:** $[a_1, b_1] \times [a_2, b_2] = [\min(S), \max(S)]$ where $S = \{a_1 \cdot a_2, a_1 \cdot b_2, b_1 \cdot a_2, b_1 \cdot b_2\}$
- **Division:** similar to multiplication, taking into account that division may be undefined if the interval contains zero

The Convergence Problem

An important challenge of interval analysis is guaranteeing the termination of the fixed-point algorithm. Consider the following example:

```
int x = 0;
while (x < 100) {
    x = x + 1;
}
```

Without additional mechanisms, the analysis would produce the following sequence of iterations:

- Iteration 1: $x \mapsto [0, 0]$
- Iteration 2: $x \mapsto [0, 1]$
- Iteration 3: $x \mapsto [0, 2]$
- $\vdots$
- Iteration n: $x \mapsto [0, n-1]$

This sequence does not converge in finite time, since the interval grows indefinitely. To solve this problem, **widening** techniques are used.

Widening

The widening operator ∇ forces convergence by accelerating the stabilization of intervals that grow indefinitely. Basically, to implement it we define fixed bounds within which our variables will move. For example: $\{-\infty, 0, 1, +\infty\}$. Then, depending on the abstract operation (addition, subtraction, etc.), we collapse the value towards the safest value.

Using the previous example, widening would quickly converge to the following mapping:

- Iteration 1: $x \mapsto [0, 0]$
- Iteration 2: $x \mapsto [0, 1]$
- Iteration 3: $x \mapsto [0, +\infty]$ (applying widening)

The Problem with Widening: "Shooting Over the Target"

Although widening guarantees convergence, it can be too aggressive and produce imprecise results. The problem arises when widening "overshoots" the actual value the variable can reach, mapping to infinity too quickly when a variable exceeds the program constants.

Consider this example:

```
public static int func() {
    int y = 0;
    int x = 7;
    int k = 3;
    int one = 1;
    x = x + 1;  // x = 8
    while(k > 0) {
        x = 7;        // x is reset to 7
        x = x + 1;    // x = 8
        y = y + 1;    // y is incremented
        k = k - one; // k is decremented
    }
    return x;
}
```

Why `one` and not `k--`? With widening, each integer constant of the program is replaced by the closest bound in B (`IntervalFinder.findBestSingleton`). In this lab $B = \{-\infty, 0, 1, 3, 7, +\infty\}$, so the constant -1 becomes 0. The Java compiler may turn `k--`, `k -= 1` and even `k = k - 1` into `k = k + (-1)`, which in Tai-e's IR reads `k = k + %ci-1`: the analysis would then compute `k = k + 0`, and k would stay $[3, 3]$ instead of decreasing. Subtracting a variable that holds 1, a bound in B, avoids it. Keep this in mind if you write your own targets: with widening, step loop counters by a variable, not by a constant that is not in B.

Analysis with widening:

- $x \mapsto [7, +\infty]$ (because x takes the values 7 and 8, and widening extends it towards $+\infty$ since 8 is not a program constant)
- $y \mapsto [0, +\infty]$ (because y grows indefinitely according to widening)
- $k \mapsto [-\infty, 3]$ (because k decreases indefinitely according to widening)

These results are imprecise: we know that x at the end of the program should be exactly 8, but widening maps it to an infinite interval.

Narrowing: Refining the Solution

To address this problem, the **narrowing** technique is used. After reaching convergence with widening, a second phase is applied that refines the result using the original ("naive") rules of the analysis without widening.

Applying narrowing to the example:
After widening: $x \mapsto [7, +\infty]$, $y \mapsto [0, +\infty]$, $k \mapsto [-\infty, 3]$
Narrowing applies the original rules iteratively:

- At the end of the program: $x \mapsto [8, 8]$ (it is refined because x always ends up being 8)
- y and k keep their infinite intervals because narrowing cannot determine how many times the loop is executed

In this way, narrowing makes it possible to recover part of the precision lost by widening, especially for variables such as x that have a predictable behavior at the end of the program.

Implementation

You work on the same framework as in HW1 (Tai-e; see `STUDENTS.md` in the repo). The analysis extends Tai-e's `AbstractDataflowAnalysis`, and works on Tai-e's IR, as in HW1. The project includes two main packages in which you will work:

Package `intervaltargets`

This package contains the Java classes that will serve as targets for testing the interval analysis. You must not modify them.

Package `intervalanalysis`

This package contains all the classes needed to implement the interval analysis. You must complete the methods that throw an *UnsupportedOperationException*. In particular:

- **IntervalAbstractValue.java:** Represents the abstract values of the interval lattice. Defines the structure $[lower, upper]$ and the special elements BOTTOM and TOP.
- **IntervalAbstractState.java:** Represents the abstract state as a mapping from variables to intervals.

- **IntervalAbstractValueVisitorNaive.java:** "Naïve" implementation of the visitor that applies the standard rules without widening.
- **IntervalAbstractValueVisitorWithWidening.java:** Implementation of the visitor that applies widening at widening points.
- **IntervalFinder.java:** Utility for finding the best interval that fits the given boundary points. Implements the widening formula $\omega'([l,h]) = [\max\{i \in B | i \leq l\}, \min\{i \in B | h \leq i\}]$.

To check your implementation as you go, we recommend running the course's tests, or adding your own. Run them with `./gradlew test` (`gradlew.bat test` on Windows). You can also run the interval analysis on a class with `./gradlew intervalAnalysis -PtargetClass=<targetClass> -Pmode=<mode>`, where `mode` is `NAIVE`, `WIDENING` or `NARROWING`; it prints the OUT state of each statement. Note that, in practice, the naive version does not terminate on loops whose intervals keep growing, such as the one in `IntervalAnalysis3`. The course also grades your submission with additional tests on the operations of `IntervalAbstractValue`.

Submission

A complete implementation of the Interval Analysis (naive, with widening and with narrowing). The tests provided by the teaching staff must pass. You may add your own if you wish.